Taiz University

Dr. Abdullmalek Alqobaty

Associated professor

Computer Engineering

Secured Application Development

Authentication and Authorization

Authentication and Authorization

Authentication and Authorization

In this lecture:

- **We will look at options for authenticating users and determining what permissions they have been given.**
  - **The most common authentication method is prompting for a username and password.**
  - **Other authentication methods include one-time passwords and biometric data.**
- **Once a user has authenticated, the application must determine what permissions that user has.** This is the authorization.
  - **We will look at various methods, including role-based authorization, JSON web tokens, and API keys.**
- **OAuth2 is a standardized protocol for authentication and authorization.** As it is a big topic, we will cover it in a next lecture.

Authentication vs. Authorization

- Authentication is the process of verifying the identity of a user.
- To achieve this, the legitimate user should be able to provide some information, such as:
  - A username and password
  - One-time password
  - A public-private key pair
  - Biometric data (e.g., fingerprint or face recognition)
  - Combination of all of these
- Two-factor authentication (2FA) authentication are examples of the latter.
- In 2FA, one factor is something the user knows. The other is something the user has.
  - The factor the user knows is commonly a *password*.
  - The factor the user has may be a device that generates a one-time password, a biometric feature such as a fingerprint, or a cryptographic signature.

Authentication vs. Authorization

- Authorization, is the process of verifying that user has permission to access a certain resource (application, file, function, etc.)
- Methods of authorization include:
  - Role-based authorization
  - JSON Web Tokens (JWTs)
  - API keys
  - OAuth2
- We will look at each of the preceding authentication and authorization methods in this lecture and the next.

Username and Password Authentication

- **Username and password authentication remains the most common choice for websites.** For this reason, it is supported by popular browsers and web servers and also provided by WAFs. **We will look at web browser and server support for authentication as well as the form-based method used by frrameworks and other WAFs.**

HTTP Authentication:

- **The HTTP specification**, in RFC 2617 and RFC 7616, **defines HTTP headers for authentication**. **These are supported by servers such as Apache and Nginx as well as common browsers. The header**

  WWW-Authenticate

  is sent by the server to ask the client to ask the user to authenticate. The client prompts for a username and password and sends them back in a new request with the header

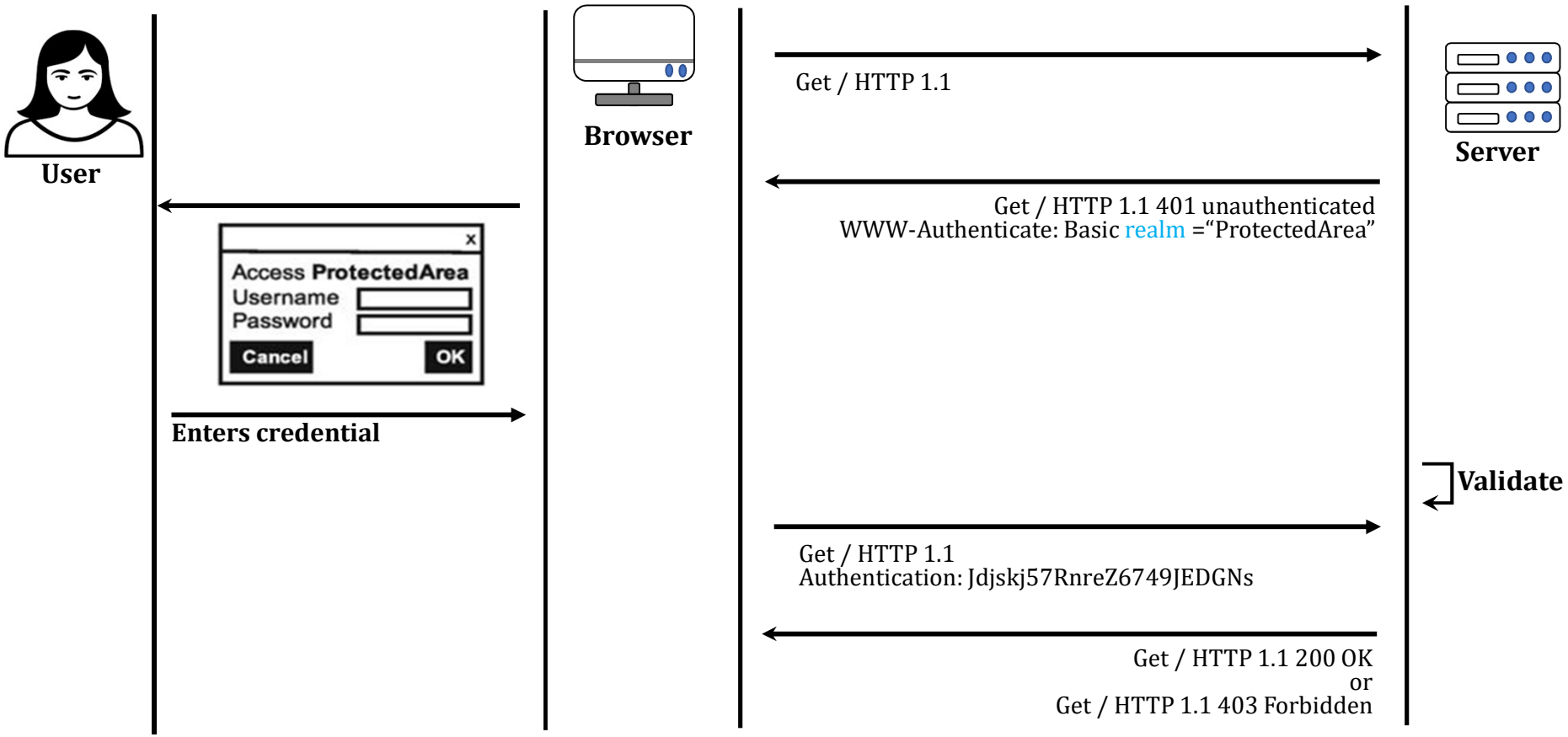
  Authorization

- **The HTTP specification supports a number of types of authentication with this method. We will look at *Basic* and *Digest* here. We will examine a third option, *Bearer*, later on.**

Basic Authentication

- **HTTP Basic Authentication is illustrated in Figure 1.**
  - **The user requests a page / that is protected by Basic authentication.**
  - **The server responds with 401 Unauthorized and includes the header**
    - **WWW-Authenticate: Basic realm="ProtectedArea"**
    - ***instructing the client that it needs to provide credentials to access the page.***
  - **The realm** is a name of the developer's choosing. **Credentials the user provides will be valid for pages with the same realm.**

Basic Authentication



Basic Authentication

- **When the browser receives a page with the WWW-Authenticate header, it prompts for a username and password in a pop-up dialog.**
- **The browser sends this back in the Authorization header by concatenating the username, a colon, and the password and then Base64-encoding them:**
  - **Base64 (*username:password*)**
  - **If the credentials match an entry in the server's database, it returns the requested page with a 200 OK response.**
  - **If they do not, the server returns 403 Forbidden.**
- **The way in which credentials are stored server side depends on the web server.**
  - For Apache, the directories that are protected by Basic authentication, along with the name of a file containing the usernames and passwords, are defined in the site configuration file, **as shown in the following example:**

Basic Authentication: Example

```
<VirtualHost *:80>
    ServerName example.com
    DocumentRoot /var/www/html
        <Directory "/var/www/html/protected">
            AuthType Basic
            AuthName "ProtectedArea"
            AuthUserFile /etc/apache2/.htpasswd
            Require valid-user
        </Directory>
</VirtualHost>
```

Basic Authentication

- Each entry in the password file contains a username, followed by a colon, followed by a hashed password.
  - Depending on the OS, the hash may be bcrypt, MD5, SHA-1, CRYPT, or simply plaintext.
- The easiest way to create them is with the htpasswd command.
  - An example with MD5 is

```
> htpasswd -nbm alice verysecretpassword alice:$apr1$ETXkAj.d$iFyxGnsDFIE.Dq5bRUjMR/
```

  - The -n option outputs the result to standard output instead of writing it to a file.
  - The -b takes the plaintext password from the command line instead of prompting for it interactively.
  - The -m option is for MD5.

Digest Authentication

- HTTP defines a second type of authentication, called Digest, to mitigate against disclosure of the password through a man-in-the-middle attack. It achieves this by ensuring the password is not sent as plaintext.
- The specification supports a number of options, making the syntax quite elaborate. We will only look at one example, adapted from one given in RFC 7616.
- Digest authentication supports MD5, SHA-256, and SHA-512-256 (SHA-512 truncated to *256 bits*)
- An example response with WWW-Authenticate requesting MD5 is

Digest Authentication

- **An example response with WWW-Authenticate requesting MD5 is**

HTTP/1.1 401 Unauthorized

WWW-Authenticate: **Digest**
realm="http-auth@example.org",
qop="auth",
algorithm=MD5,
nonce="7ypf/xlj9XXwfDPEoM4URrv/xwf94BcCAzFZH4GiTo0v",
opaque="FQhe/qaU925kfnzjCev0ciny7QMkPqMAFRtzCUYo5tdS"

- **The qop parameter stands for *quality of protection*. The value auth means authentication (an alternative is auth-int, which means authentication with integrity protection).**
- **Depending on the qop, the credentials are sent with different formats.**

Limitations of HTTP Authentication

- The primary goal of Digest authentication was to avoid sending the password in plaintext, mitigating the risk of man-in-the-middle attacks.
  - However, it comes at a cost. The server has to have the plaintext version of the user's password in order to reconstruct the response hash (it is a function of the nonce as well as the username, so it cannot be stored in hashed form).
  - This means that if the password file is compromised, an attacker has the plaintext version of all the users' passwords. The threat is shifted from man-in-the-middle to compromising the server.
- HTTPS is now seen as the best defense against man-in-the-middle attacks, and in practice, this mitigates the risks associated with sending plaintext passwords as well as or better than Digest authentication.
  - This means Basic authentication can be used and passwords can be stored on the server in hashed form.
  - For this reason, Digest authentication is rarely used, and we will not examine it in more detail.

Limitations of HTTP Authentication

- In practice, Basic authentication is also used rarely. It has a number of limitations, including the following:
  1. The username/password popup does not integrate well with an application's look and feel.
  2. The applications security configuration is dependent on the choice of web server.
  3. User management is less flexible because usernames and passwords are stored in a file.
  4. Users cannot be given roles with differing levels of access.
- Some of these limitations can be reduced by configuring the server to pass the headers to the application for processing, rather than performing authentication itself.
- However, most developers choose an alternative, form-based authentication, which we will look at next.

Form-Based Authentication

- **Form-based authentication is performed by the application, not the web server. When a user visits a page that requires authentication, and no credentials have been provided, the application redirects the user to a login page. The process is illustrated in Figure 10-2.**

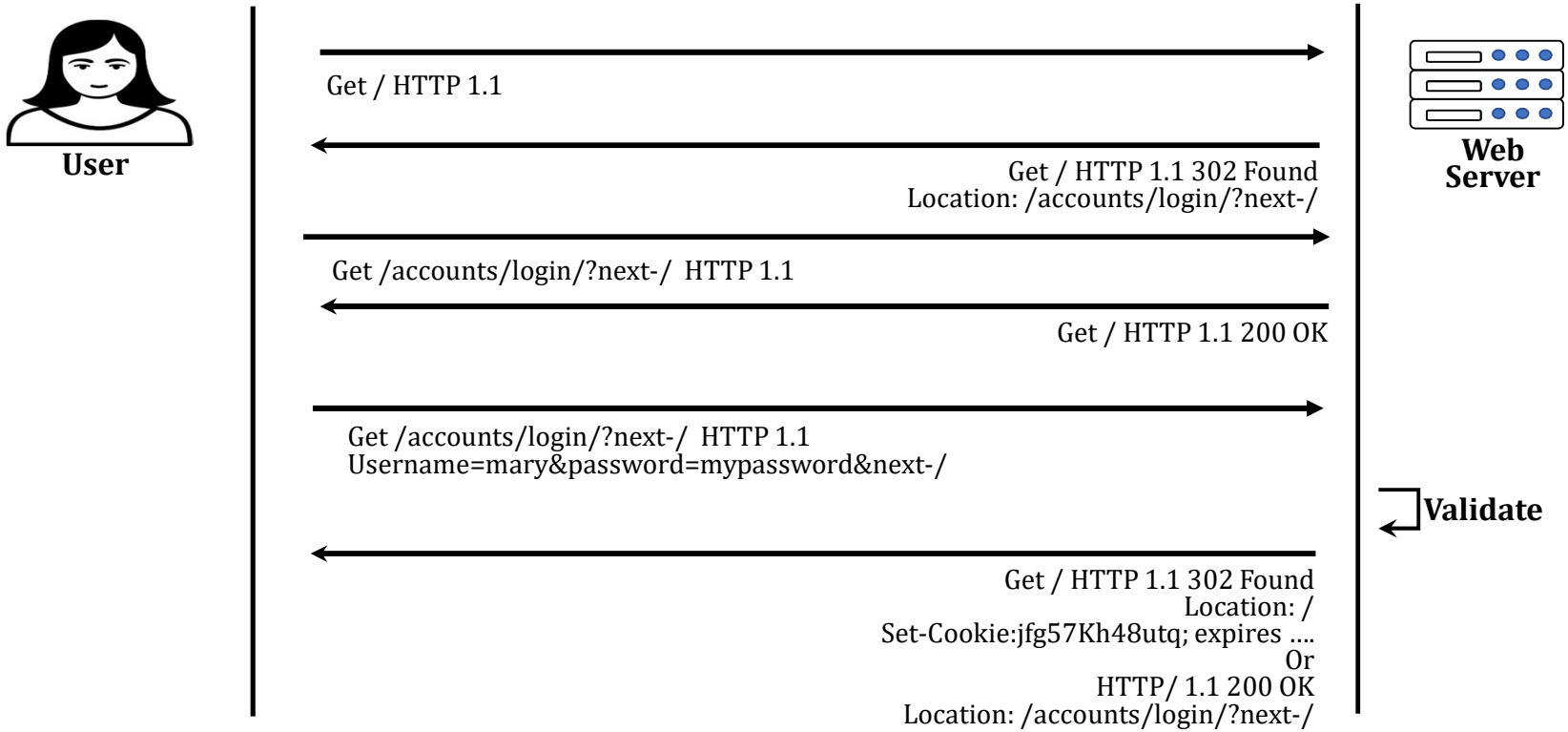


Figure 10-2 Form-based authentication

Form-Based Authentication

- The login page the user is redirected to contains a form to prompt them for their username and password.
  - Submitting this form sends the username and (plaintext) password to the server. If they match an entry in its user database, the server sends the user to their originally requested page, usually with a Found redirect. At the same time, a session cookie is set with the Set-Cookie header.
  - If the username and password are incorrect, the user returns to the login page with an additional error message.
- The next time the user requests a page, the session ID is sent to the server in the Cookie header. The server validates it against its session table. If it is valid, the user is taken directly to their requested page. If not, the user is prompted to log in again.

Disadvantages of Form-Based Authentication

- One disadvantage of form-based authentication is that a failed authentication attempt is not clearly reflected in HTTP response headers. If a user enters invalid credentials, the server responds with an error message in HTML, but the response code is still 200 OK. This makes monitoring failed attempts harder.
- Form-based authentication can be inconvenient for single-page applications and API calls. If the user has not authenticated, or the session ID cookie has expired or been deleted, the user is redirected to a login page.
  - For a single-page application, this means redirecting away from the application.
  - For API calls, this means returning an HTML form instead of, for example, JSON data.
- An alternative is to send a username and password, or username and API key, with each request. API keys are described later in this lecture.
- The OAuth2 protocol is another way to provide authentication for REST API calls. OAuth2 is described in the next lecture.

One-Time Passwords

One-Time Passwords

- One-time passwords (OTPs) become invalid once they have been used once. After an OTP has been used to log into a server, the server no longer accepts it as a valid password.
- OTPs are often used together with another form of authentication, such as username/password, as a two-factor authentication process.
  - There are two popular algorithms for generating OTPs.
    - HMAC-Based One-Time Passwords (HOTPs) and
    - Time-Based One-Time Passwords (TOTPs).
  - There are also two ways of delivering them to the user.
    - They are either generated by the server and sent to a device registered to that user (SMS) or
    - The user has a device that can independently generate an OTP that matches the next one the server expects. In the latter scenario, the OTP is never sent over a network.

One-Time Passwords

- When talking about OTPs, we talk about a token service and validation service. The token service issues the OTP, and the validation service validates it.
  - If an application sends an OTP via SMS, and the user logs into the same application with it, then both the token and validation services are the same.
  - A token service may be an application running on a user's smartphone, or a dedicated piece of hardware. In this situation, the token and validation services are different.

HMAC-Based One-Time Passwords

- **HOTPs are defined in RFC 4226.**
  - **They are based on a symmetric key, known only to the token and validation services, and an increasing counter value. The counter is kept in sync between the token and validation services.**
  - **HMAC is used together with a hashing algorithm such as SHA-1 or SHA-256 to make a hash that is a function of a secret key as well as the plaintext. HOTPs use the same algorithm.**
  - **Given the symmetric key K, the counter C, an HMAC algorithm H(·) (HMAC-SHA-1 is often used), and a number of digits d, the HOTP is defined as**

$$HOTP(K, C) = Trunc_d(H(K, C))$$

That is, the HOTP hash is a function of the symmetric key and the current counter value. $Trunc_d(h)$ is a function that truncates the hash to d digits. Six digits is common for HOTPs. For SHA-1, d must be less than or equal to ten.

HMAC-Based One-Time Passwords

- By using HMAC, the HOTP is cryptographically secure—an attacker cannot reproduce it without the secret key. Each time an HOTP is generated, the token service increments its counter by one. Each time the validation service consumes it, it also increments its counter.
- It can happen that the validation service's counter lags behind the token service's. This happens if tokens are generated but not used. Therefore, a synchronization process can be added, allowing the validation service to look ahead for a small number of counter values. If this fails to achieve a matching HOTP, the two services must be resynchronized manually.

Time-Based One-Time Passwords

- **TOTPs are similar to HOTPs. The difference is a time code replaces the counter.** They are defined in RFC 6238 [19].
- **To calculate the counter *C*, TOTP introduces two parameters:** T_0, which is the Unix time to start counting from (defaulting at 0), and T_X, which is the number of seconds between counter values, defaulting at 30. The counter C is defined as

$$C = \left(\frac{T - T_0}{T_x}\right)$$

  where T is the current time in seconds. A new counter value will be produced every T_X seconds.
- **TOTPs have the advantage of not losing synchronization between the token and validation services, unless their clocks significantly differ. A new TOTP replaces the old one every T_X seconds regardless of whether one is requested or not.**
- Because clocks can differ slightly, **some validation services accept a *C* one greater than and/or less than its own value.**
- **TOTPs created this way are the most common ones used due to their simplicity** and the existence of mobile apps that implement them, such as Microsoft Authenticator and Google Authenticator (we look at Google Authenticator in the following).

Sending OTPs via SMS

- SMS delivery of OTPs used to be common and is still used.
  - The advantages are it avoids the problem of counters losing synchronization and doesn't need the user to have any special device or application installed, other than a mobile phone.
- However, SMS-based delivery has been found to be insecure, *and NIST, in its Digital Identity Guidelines, recommends against using it without further mitigating defenses*.
  - Firstly, there is a physical risk of an attacker removing a SIM card and inserting it in their own device.
  - Secondly, phishing attacks may be used to obtain sufficient information to convince a mobile service provider to transfer a victim's phone number to the attacker's device.
    - This tactic was used against Twitter CEO Jack Dorsey in 2019 and used to compromise his own Twitter account.

Sending OTPs via SMS

- Malware also exists, which, when installed on a victim's device, can access SMS messages.
- Finally, the protocol used by most carriers to communicate between switches, known as Signaling System 7 or SS7, is sufficiently insecure that SMS messages can be intercepted.
  - The method is similar to man-in-the-middle and can be achieved with a standard Linux computer and the SS7 SDK.3 A high profile attack was executed in 2017, impacting German bank accounts.

- The alternative is to use an application that implements the HOTP or TOTP protocol such as *Google Authenticator*.

Google Authenticator

- Google Authenticator is a token service application, available for Android and iPhone, that implements the HOTP and TOTP protocols.
- It is released under the Apache License 2.0 and available as open source on GitHub.
  - It's original purpose was to enable users to log into their Google accounts using 2FA. However, it is sufficiently generic to also be useful as a token service for other applications.
  - It can generate HOTPs or TOTPs (the default is TOTPs).

Installing the Secret Key

- To generate valid tokens, Google Authenticator must have the same secret key the validation service (web application) uses. This is done in a registration stage, and the key is stored by the token service.
- The simplest way of sending a secret key from the validation service to the token service is for the validation service to print the key on the screen and for the user to manually type it into the token service. This is awkward for the user and error-prone.
- Google, as part of its Authenticator, has proposed a URI format for expressing secret keys, along with the account name and other data needed by the token service. The general form is

otpauth://type/label?parameters

A complete example of an otpauth URI for a TOTP key is:

otpauth://totp/ACME%20Co:john.doe@email.com

No parameters.

Authentication with Public-Key Cryptography

Authentication with Public-Key Cryptography

- The SSH protocol uses a public-private key pair for authentication. TLS/SSL uses the same technique to authenticate servers.
  - The Web Authentication API, or WebAuthn, is a set of classes implemented in JavaScript that can be used to add authentication to web applications.
  - The reason for being JavaScript is to be able to use resources on the user's device.
- WebAuthn defines three roles:
  - The Server (relying party) is the entity that creates credentials and uses them to validate the user.
  - The Client is the JavaScript application that is running in the user's browser.
  - The Authenticator stores the credentials. This may be embedded in the browser, part of the OS, or an external device such as a USB dongle.
- WebAuthn also defines two processes:
  - Registration is the process of creating credentials to be stored by the Authenticator.
  - Authentication is the process of logging in using the stored credentials.

Registration

- The registration process makes use of a private key embedded in the Authenticator that has a certificate that is digitally signed by a trusted authority.
- The algorithm uses the concept of a signed challenge:
  1. Entity A sends a challenge, which may be a random string, to Entity B.
  2. Entity B signs it using its private key.
  3. Entity A validates the signature using Entity B's public key. If It matches the challenge it sent to Entity B, and The public key has a certificate signed by an entity that Entity A trusts then Entity A is satisfied the response came from Entity B.
- The registration process is illustrated in Figure 10-6. We have drawn the Authenticator as a USB dongle though of course it may be a software component of the browser or OS.

Registration

WebAuthn

- **WebAuthn (Web Authentication API) defines specific functions for web applications to enable secure, passwordless authentication using public-key cryptography. These functions are exposed via the browser's navigator.credentials interface.**
- **The two core JavaScript functions used in WebAuthn "ceremonies" are:**
  - **navigator.credentials.create(): This function is used during the registration process to generate a new public-private key pair (credential)** and associate it with a user account.
  - **navigator.credentials.get(): This function is used during the authentication** (login) **process** to verify the user's identity by proving possession of the private key.

Registration

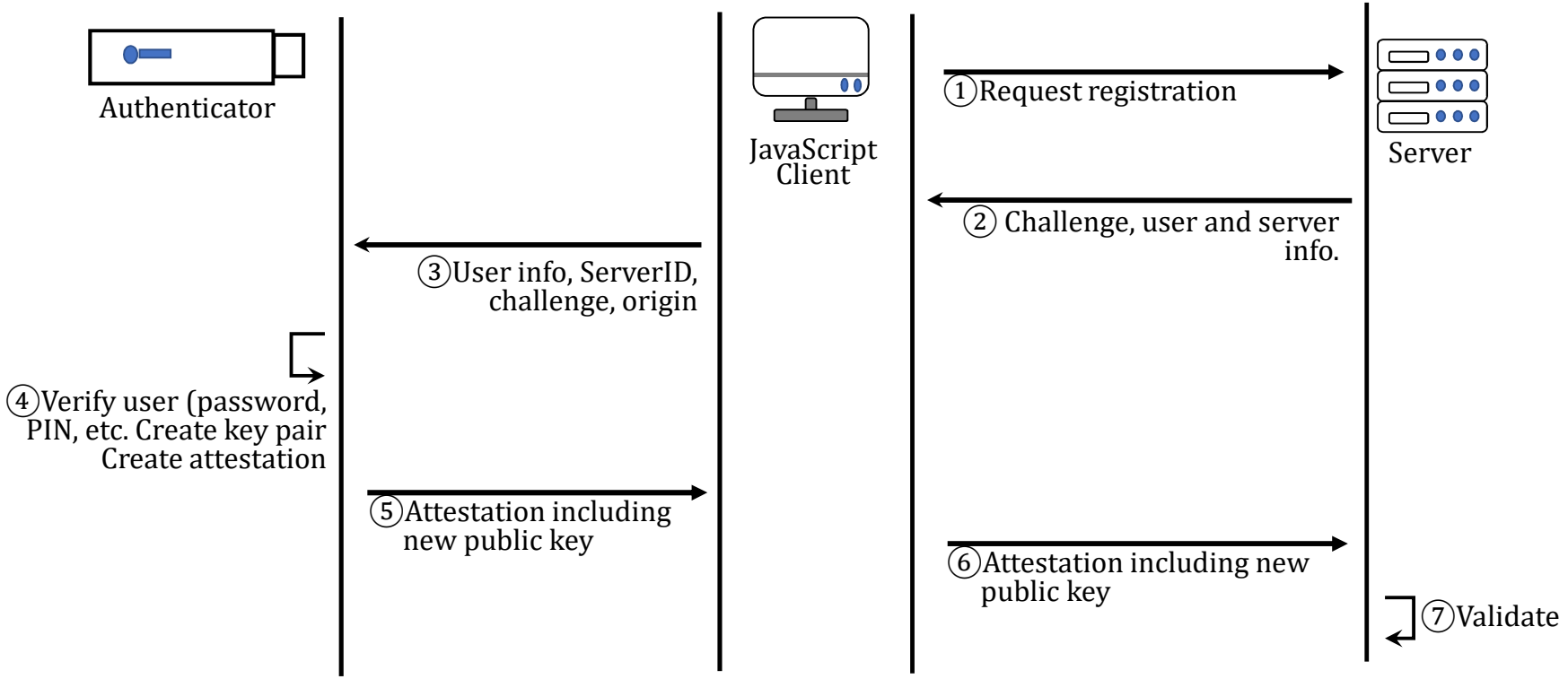


Figure 10-6 Registration using the Web Authentication API

WebAuthn

- `navigator.credentials.create()`
- **navigator.credentials.get()**

Registration

1. Step 1 in the diagram, the Client requests authentication to begin. The method for requesting this is not defined by WebAuthn but would typically be a REST API call.
2. Step 2, the Server responds with a challenge, along with its server details and the user's ID .
3. Step 3, the Client makes a WebAuthn API call to the Authenticator, sending the user info, server ID, and the origin of the request.
4. Step 4, the Authenticator validates the user by requesting a password, or a PIN, or scanning biometric data such as a fingerprint or face. Some simply require the user to have access to the device, for example, by simply pressing a button. If the validation succeeds, it creates and stores a new public-private key pair.
5. In Step 5, the public key forms part of an attestation that is signed with the Authenticator's private key so that the Server can validate it. This is sent back to the Client.
6. In Step 6, the Client application sends the attestation to the server for example, via a REST API call, containing the new public key.
7. Step 7, the server validates this object to confirm its origin and saves the public key.

Authentication

The authentication process is shown in Figure 10-7.

1. Step 1, the Client requests that authentication begin.
2. Step 2, the Server sends a challenge to verify the identity of the Authenticator when responds.
3. Step 3, the Client sends the server ID, challenge, and origin of the request to Authenticator.
4. Step 4, the Authenticator asks the user if they are happy for the credentials to be provided. If so, it fetches the corresponding private key and verifies the identity of the user (again using a PIN, password, or biometric data).
5. In Step 5, it creates an assertion including the challenge, server ID, and origin and signs it with the private key for the account. It returns this to the Client.
6. In Step 6, the Client sends it on to the Server.
7. In Step 7 , the server validates the assertion using the public key stored for the account. If it validates, it can sign the user in.

The WebAuthn protocol is complex; however, much of it is handled by the WebAuthn implement-ation itself, leaving relatively little for the application developer to implement.

Authentication

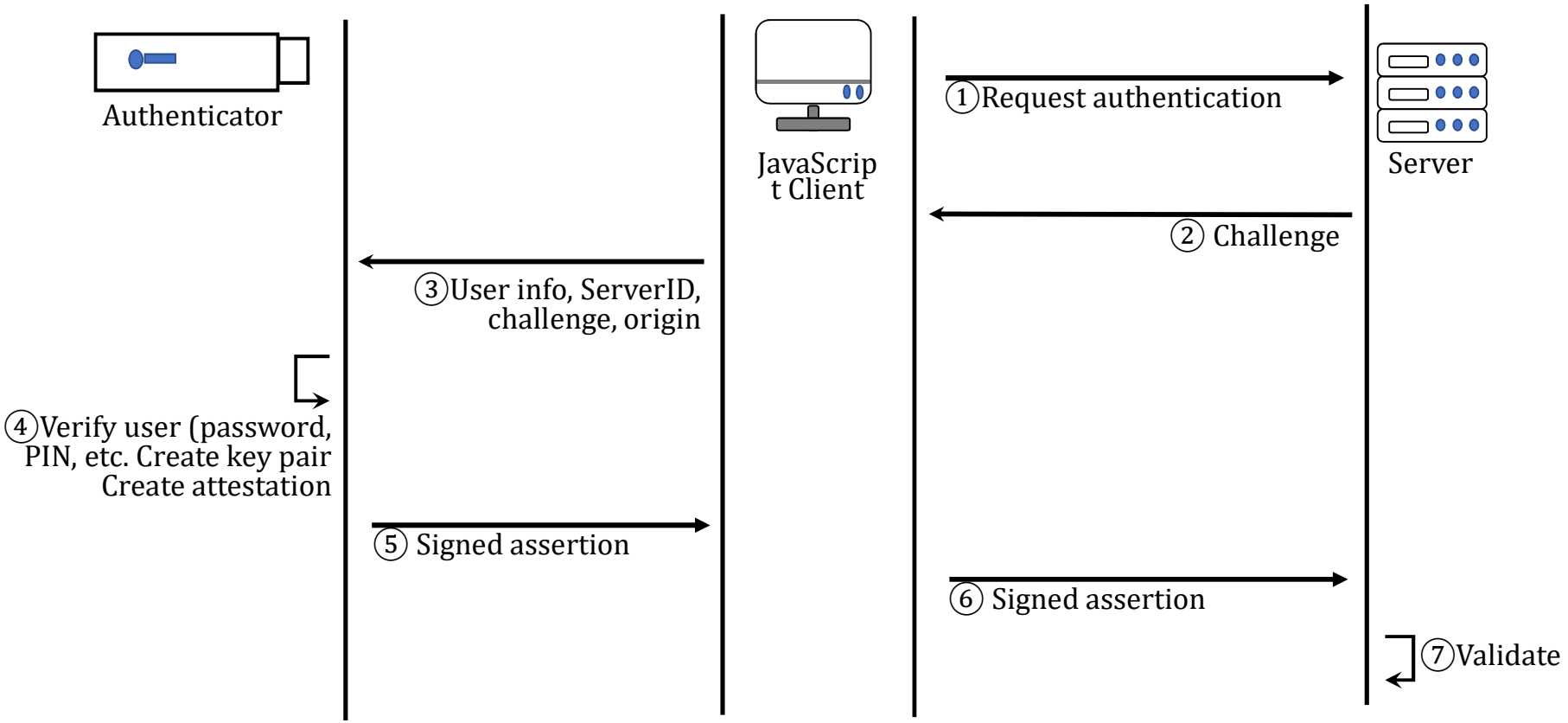


Figure 10-7 Authentication using the Web Authentication API

Biometric Authentication

Biometric Authentication

- **Biometric authentication works by capturing features of a person's body other individuals do not have. The commonly used biometric measures are fingerprints and facial features. The most common use is for reauthentication after a user has already signed in by other means (such as a username and password), but they can also be used for 2FA.**
- **Biometric authentication is now widely used in smartphones, which have dedicated scanners for this purpose. Developing biometric scanners is not trivial.**
- **Developers of biometric scanners use the term *liveness* to describe properties that differentiate a real person from a photo or mask.** For example, some fingerprint scanners use capacitive sensing that detects electrical conductivity rather than capturing an optical image.
- **Apple's Face ID facial recognition uses an infrared light source and receiver to create a depth map of a face. It also detects if the subject is awake to prevent using on a sleeping victim.**
  - **A Vietnamese security company Bkav found they could fool Face ID by using a 3D scanner to make a depth map of the target's face, constructing a 3D-printed wireframe, employing an artist to tweak features, and then imposing photographs of the target user.**
  - **Another liveness measure some scanners use is capturing a sequence of photos to detect movement, such as eyes blinking.**

Biometric Authentication

- The Web Authentication API can be used to authenticate with biometric scanners on smartphones.
  - The JavaScript implementation in the main browsers on iPhone and Android interface with the devices' biometric APIs. The biometric scanning takes the place of password or PIN entry.
- Refer to an exercise in the book. The exercise is more complex as it needs more than just the Vagrant VMs to run. Firstly, you will need a device with fingerprint or face authentication supported by its OS and a web browser.

Role-Based Authorization

Role-Based Authorization

- We now turn our attention to authorization: determining what a user has permission to do, once they have logged in.
- In role-based authorization, permissions are associated with roles, and users are assigned those roles. The role may be a property on the user table, in which case the relationship between a role and user is one to one. Alternatively, you may assign roles in a many-to-many relationship, as shown in the relationship diagram in Figure 10-12.
- Once a role model is defined, you can conditionally allow or disallow functionality based on the logged-in user's membership of a role. For example, you can create a role called admin.

Role-Based Authorization

- For example, you can create a role called admin. If a logged-in user attempts to access the administration pages of a web app, you can check if the user is a member of the admin role and return 200 OK or 403 Forbidden accordingly.

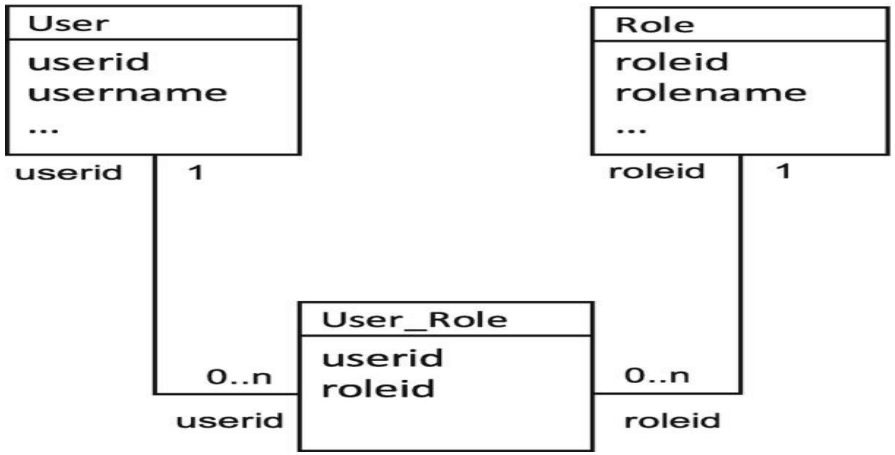


Figure 10-12 Simple many-to-many role-based authorization model

Role-Based Authorization

- **Complex applications may need a more complex authorization model.** For example, you may have different levels of membership such as Basic, Pro, and Enterprise. You may want those three groups have access to different components of the application.
- **It is error-prone to assign permissions to individual users.** Instead, we can assign users to groups and associate permission with groups rather than users, as shown in Figure 10-13. Using this model, you can change what roles a Pro user has access to, for example, without having to create a new association for every user.

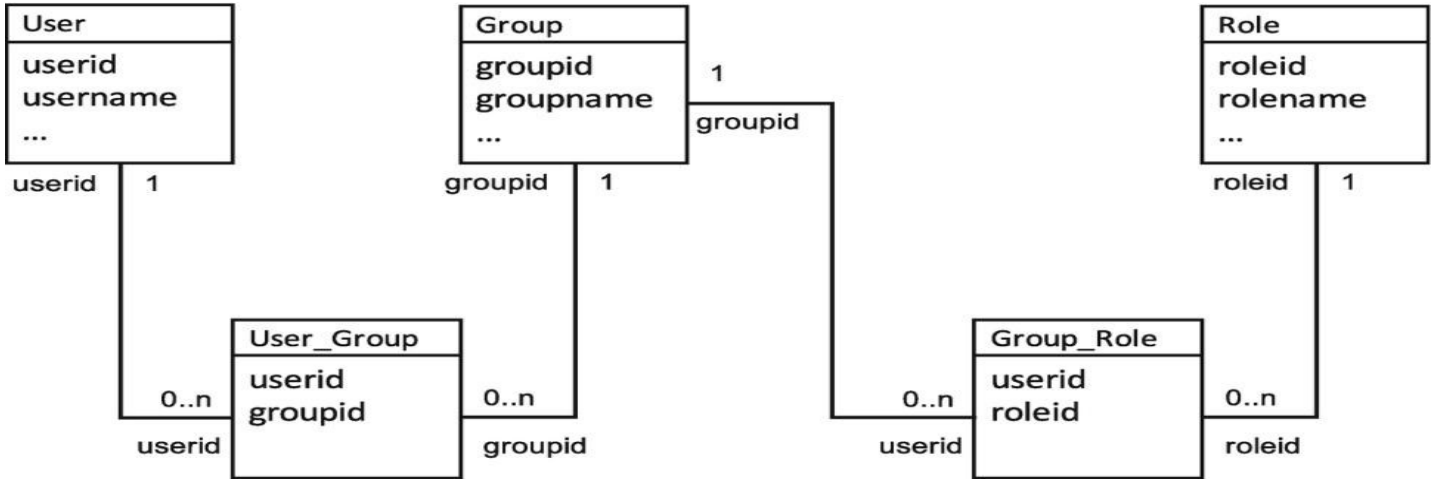


Figure 10-13 Role-based authorization with groups

JSON Web Tokens

JSON Web Tokens (JWTs)

- **JSON Web Tokens, or JWTs, are a standardized syntax for transmitting data between two parties. They are signed to guarantee authenticity and optionally encrypted. They are useful for transmitting authorization data. (JWTs are defined by RFC 7519. The jwt.io website).**
- **JWTs are designed to be small. They consist of three parts, each separated by a dot:**
  1. **a header,**
  2. **a payload,**
  3. **and a signature.**

  The final token looks like xxxxx.yyyyy.zzzzz.
- **Example JWT: A complete, signed JWT string looks like this:**

  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiYWRtaW4iOnRydWV9.dyt0CoTl4WoVjAHI9Q_CwSKhl6d_9rhM3NrXuJttkao

JSON Web Tokens (JWTs)

1. **The header is a JSON string with the form**

```
{
    "alg": " HS256 ",
    "typ": "JWT"
}
```

- **The *algorithm* value denotes the algorithm used for the signature. HMAC with SHA (HS256, HS384, etc.) and RSA with SHA (RS256, RS384, etc.) are common choices.**
- **The header is then Base64Url-encoded. This is identical to Base64 encoding but with - in place of +, _ in place of /, and =, which is the trailing padding character, either omitted or URL to make Base64 compatible with URL syntax.**

JSON Web Tokens (JWTs)

2. **The payload consists of claims, also as JSON. Claims are statements about an entity such as a user.**

   There are three types of claim:

   1. **Registered claims: Predefined claims defined in the JWT standard. Common registered claims are as follows:**
      - **sub: The subject, such as the username**
      - **iss: The issuer of the JWT**
      - **exp: The expiry time, represented as the number of seconds since midnight on Jan 1, 1970**
      - **aud: The audience, or intended recipient, usually its base URL, for example, https://example.com**
   2. **Public claims: Claims that can be freely chosen by the implementer but that should be defined in the IANA JSON Web Token Registry11 or prefixed to avoid collision**
   3. **Private claims: Claims that the implementer is free to define, for example, details about a user such as the real name**

JSON Web Tokens (JWTs)

- **An example payload is**

```
{
    "sub": "alice",
    "exp": 1672531199,
    "name": "Alice Adams",
    "type": "member"
}
```

The payload is Base64Url-encoded.

3. **To create the signature, the signature is used to verify the token's integrity and ensure it hasn't been tampered with. It is created by combining the encoded header, the encoded payload, and a secret key using the specified algorithm.**

HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret_key)

- **Since JWTs are signed, the client cannot alter permissions contained in them.**
  - **For example, if Alice tried altering the preceding payload, changing "type" to "admin", it would no longer match the signature, and the server would know it has been altered. It also helps validate that the claim is from a trusted entity.**

Storing and Transmitting JWTs

- Once a server has authenticated a user, it sends the client the JWT. This can be by setting a cookie with Set-Cookie, in an HTTP header, or by sending it in the response body, for example, in JSON.
  - If sent as a cookie, the browser sends it in subsequent requests in the Cookie header.
  - If sent in the response body, the browser needs to store it, either in a JavaScript variable, in HTML5 session storage (window.sessionStorage), or HTML5 local storage (window.localStorage). In these cases, the client should sent it to the server as an Authorization: Bearer header:

    Authorization: Bearer jwt-token
- We saw Authorization: Basic and Authorization: Digest in the beginning of this lecture.
- Bearer is a scheme that takes an application-specific token as its value.

Storing and Transmitting JWTs

- JWTs are vulnerable to the same attacks as session IDs. As discussed cookies can be vulnerable to XSS if httpOnly is not set.
- They can also be vulnerable to CSRF, as discussed, if CSRF tokens and the SameSite attribute are not used correctly.
- If JWTs are stored in HTML5 session or local storage, they can be vulnerable to exfiltration by scripts that can also read the storage. For this reason, sensitive data such as authorization tokens should not be stored with this method.
- Storing JWTs in as cookies or in a JavaScript variable is the safest option.

Revoking JWTs

- An advantage of JWTs is that the server does not have to query a database to determine if a user is authorized. It does not even need to store session state as all the data it needs to determine the user's credentials are in the JWT and signed by its own key. This makes them useful for single-page web applications that make extensive use of API calls.
- However, if no server-side state is stored, the JWT authorization cannot be revoked. One solution to this is to make the expiry short, typically a few minutes, and also issue a refresh token. This is also a JWT, but with a longer expiry, say, a month or more.
- The client sends the JWT authorization token in requests to the server until it reaches expiry. When that happens, it sends the refresh token instead. The token is stored in a special table if it has been revoked.
- When the server receives the token from the client, it checks to see if it is in this table. If not, it creates and returns a new authorization token. If it is revoked, the server responds with 403 Forbidden.
- The process is illustrated in Figure 10-17.

Revoking JWTs

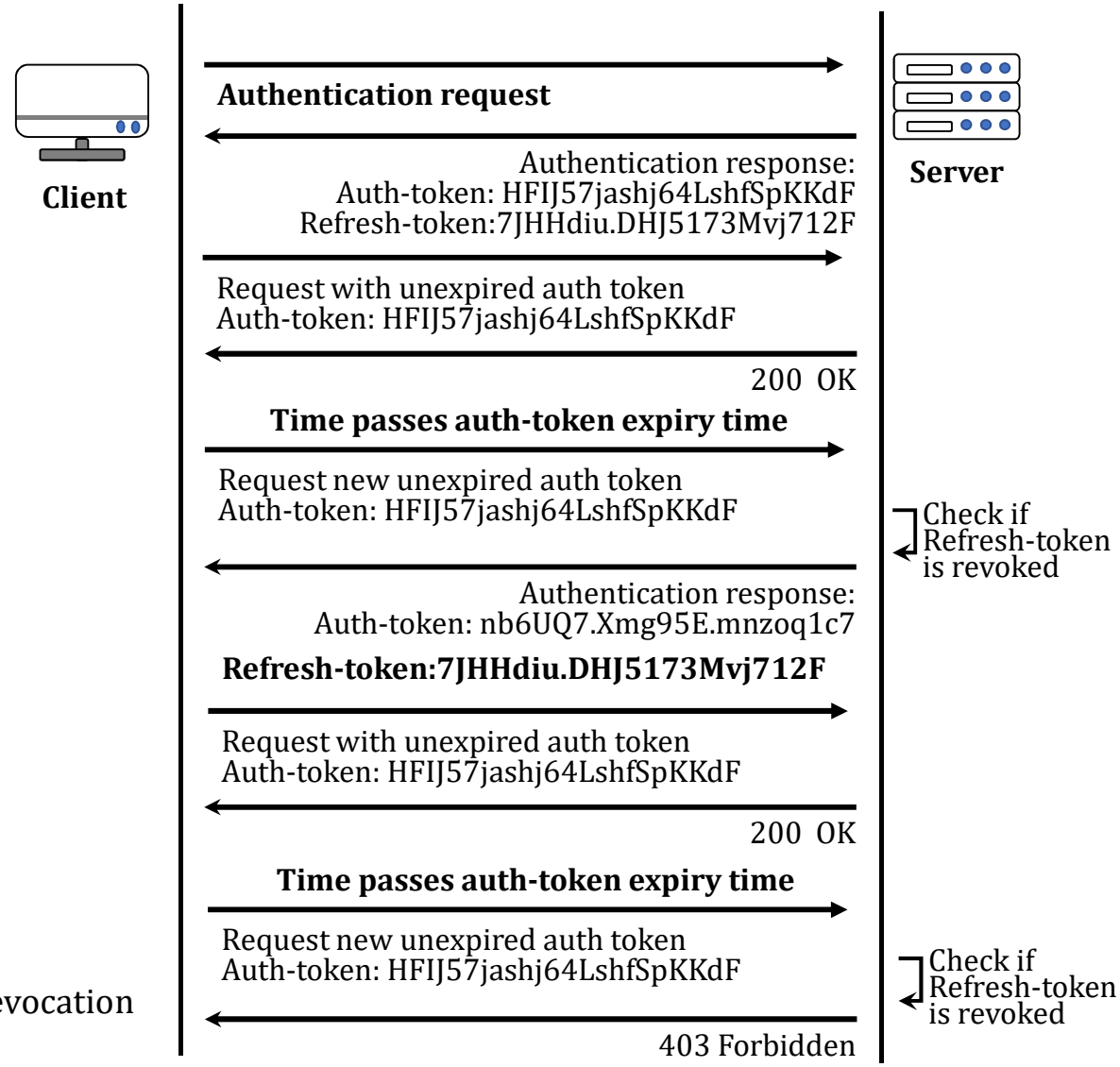


Figure 10-17 JWT refresh tokens to enable revocation

API Keys

API Keys

- An API Key is a unique code that identifies and authenticates applications or developers accessing an Application Programming Interface (API), acting like a password to control usage, track data, and manage permissions for services.
- We saw that form-based authentication with session IDs is inconvenient for REST APIs as the client has to handle redirects to an HTML form if the session ID has expired.
  - An alternative is to always send a username and password with each request. However, this means the password must be stored by any application making the REST API calls.
  - Not only this is risky for the account owner, but it may grant more permissions than the client actually needs. API keys are designed to address these issues.
- A user or developer creates an API key for the application they want the REST API to access on their behalf.
- The developer of the REST API can allow the user to select from a number of roles, for example, read-only vs. read-write. To access the API, the newly created key is sent instead of the user's password.

API Keys

- Imagine a user, Alice, has an account with a service, weatherfor.com, that provides an API to get weather forecasts. The API call

  https://weatherfor.com/api/forecast/New+York

  returns a JSON string with the current forecast for New York. Alice wants to develop an application that uses this API. She visits weatherfor.com, logs in, and goes to the page where she can create API keys.
- The weatherfor.com site will ask Alice to enter an application name which she can use later to refer to her new API key. It will create her a cryptographically secure code. This may be a random string. Alternatively, like Django session IDs, it may be information about the user and/or application signed with a secret key.
- The server displays the new API key so that Alice can copy it into her application. It also stores it in a table associated with Alice's ID. When Alice uses the key in her application, weatherfor.com can determine which user it belongs to and whether that user is authorized to access the API.

API Keys

- Imagine another example where Alice is developing a social media site, homies.com.
  - She creates an API so that other developers can create applications that interface with her site. She knows her users may want third-party applications to have permission to perform some operations, for example, read friends' posts and be able to prevent them creating posts or reading their personal details.
  - Therefore, Alice defines several different roles: read-posts, create-posts, read-personal-details, etc. Now her customers can create an API key with just the read-posts permission and use that API key in the third-party application, confident that it will not be able to read their personal details.
- The advantages of API keys over usernames and passwords are as follows:
  - API keys can have restricted permissions.
  - Users do not have to enter their personal passwords into applications.
  - They are algorithmically generated, so they do not suffer from bad password selection policies as user-generated passwords often do.

API Keys

- API keys are analogous to passwords. Many sites store them as plaintext, which is rather like storing passwords as plaintext. A better practice is to hash them (with SHA-256, PBKDF2, etc.) and store the hash instead. When an API is called with an API key, the server hashes it and compares it with the hashed version in the database.
- The disadvantage of this approach is that if Alice loses her API key, she cannot retrieve it from weatherfor.com. She must instead create a new one and update her application's configuration.
- Unlike JWTs, API keys are preshared, meaning the API does not have to support sending the API key from the server to the client.
  - To send a new key from the server to the client, the user usually copies and pastes it.
  - To send it from the client to the server, it can be included in the Authorization: Bearer header or sent in the request body (as a POST parameter or as part of a JSON string).

Sending API Keys

- Unlike JWTs, API keys are preshared, meaning the API does not have to support sending the API key from the server to the client.
  - To send a new key from the server to the client, the user usually copies and pastes it.
  - To send it from the client to the server, it can be included in the Authorization: Bearer header or sent in the request body (as a POST parameter or as part of a JSON string).
- API keys have some limitations. If a third-party developer, Bob, wants to interface with Alice's homies.com site, Bob's users must log into homies.com, create an API key, set the correct permissions, and then paste it into Bob's application. This is a technical process that puts off some users. If Bob attempts to automate the process, he must ask his users to enter their homies.com username and password. Users may be hesitant to give a third-party application their password—after all, this is what API keys were supposed to avoid.
- The alternative is to have a single API key configured in the application at server level. This is fine for situations where only anonymous access is needed, like in our weatherfor.com example, but the application will not be able to perform operations on behalf of a logged-in user.
- OAuth2 was designed, among other goals, to address these limitations.